Hierarchical Memory for Behaviorally Grounded Generative Agents

Manal Sultan

April 27, 2026

Abstract

This report presents a research oriented memory architecture for behaviorally grounded generative agents. Inspired by Soon Jung Park's 2025 dissertation *Generative Agent Simulations of Human Behavior*, the project investigates whether hierarchical memory structures with provenance preserving summaries can better support targeted retrieval and compressed social state representation than flat episodic memory alone. The system organizes timestamped observations into a multilevel memory graph, builds higher level summaries over related episodes, supports both flat and hierarchical retrieval, separates answer generation from verification, and exposes provenance, retrieval traces, model traces, and derived social state abstractions through an inspection interface. Evaluation is conducted through seeded behavioral memory scenarios and a frontier sweep over retrieval policy configurations spanning canonical tasks, unseen seeds, and hard perturbations. The results indicate that hierarchical retrieval consistently improves slot and keyword recovery relative to the flat baseline while revealing a clear tradeoff between recall and token efficiency. Overall, the project contributes an experimentally grounded memory substrate for future work on generative agent reasoning, behavioral simulation, and intervention oriented scenario analysis.

1 Motivation and Research Question

The internal structures that allow generative agents to remember, synthesize, and act on experience over time are an underappreciated part of what makes them scientifically and practically useful. In Soon Jung Park's dissertation, memory serves as the substrate through which agents accumulate observations, retrieve relevant context, generate higher level reflections, and maintain coherent behavior across unfolding situations. This project implements this idea by focusing on the memory and reasoning substrate itself.

The central question is whether replacing a flat episodic memory stream with a hierarchical, provenance preserving memory structure improves retrieval quality and supports more useful higher level abstractions of social state. The hypothesis is that a hierarchy can do so by reducing the burden of reasoning

directly over an ever growing stream of raw episodes: instead of repeatedly searching leaf level events, the system can move across abstraction levels, retrieve summaries when they are sufficient, descend to supporting evidence when detail is required, and preserve the provenance needed to validate those abstractions. That same logic motivates social state abstraction, which compresses memory into intermediate forms such as commitments, revisions, tensions, relationship guidance, and likely next actions that are more useful for coherent downstream behavior.

This project therefore investigates whether hierarchical memory structures with provenance preserving summaries can better support behaviorally grounded agent reasoning, targeted retrieval, and compressed social state representation than flat episodic memory alone.

2 Context and Intellectual Lineage

The dissertation argues that social systems and human institutions embed assumptions about how people perceive, decide, and act, and that generative agent simulations can serve as a new instrument for surfacing and testing those assumptions before deployment. It frames generative agent systems as a way to explore emergent dynamics, probe counterfactual designs, and move from stylized behavioral models toward richer, language mediated agents. At the architectural level, the dissertation identifies three core capabilities that make such systems coherent: memory, understood as structured episodic records; reflection, understood as periodic synthesis over those records; and planning, understood as multi timescale guidance for action.

This project adopts that general framing while focusing on a narrower systems problem. It preserves the dissertation's broad loop of observation, retrieval, synthesis, and reuse, and it retains the claim that higher level agent cognition must remain tethered to lower level evidence if it is to be inspectable and trustworthy. At the same time, the implementation in this repository is an independent engineering interpretation rather than a literal reproduction of the dissertation's systems. The codebase introduces a typed multi level memory schema, explicit parent child support relationships, summary verification, retrieval and model traces, a social state digest, counterfactual replay, and a visual inspection interface. These design choices are intended to make the architecture legible, testable, and experimentally useful within a smaller scope.

Accordingly, this project should not be understood as a full population scale generative agent simulator. It is a memory, retrieval, abstraction, and evaluation substrate for behaviorally grounded agents. Its purpose is to make the internal structures that support coherent agent behavior inspectable and experimentally tractable, rather than to model full society scale emergence.

3 System Overview

The system implemented in this repository provides an end to end pipeline for ingesting experiences, organizing them into a hierarchical memory structure, retrieving relevant evidence, generating and verifying responses, and exposing both raw and derived state through trace and inspection surfaces. At its lowest level, timestamped experiences are written as L0 memory nodes. These nodes may represent episodes, reflections, or plans, and can carry associated metadata such as entities, topics, and source provenance. From there, the system can construct higher level summaries over related observations, producing L1 summaries that compress longer threads of experience into more behaviorally useful abstractions while preserving explicit links back to the underlying support nodes.

Once a memory hierarchy exists, the system supports two retrieval regimes. The first is a flat baseline that ranks raw L0 nodes directly according to a combination of relevance, recency, and importance. The second is a hierarchical retriever that can route queries through summary nodes, descend into supporting evidence selectively, and assemble a token budgeted context that better matches the structure of the query. Retrieved context can then be passed into a model backed answerer, and the resulting answer can be evaluated separately by an answer verifier. Both the retrieval process and the model calls are recorded as traces, allowing downstream inspection of not only what the system produced, but how it arrived there.

Around this core pipeline, the repository exposes several researcher facing and user facing capabilities. On the API side, the FastAPI application provides endpoints for single and batch memory ingestion, hierarchical and flat retrieval, summary building, refresh and staleness management, provenance inspection, timeline and tree views, social state digest generation, evaluation runs, ablations, and counterfactual replay. On the interface side, the inspection UI visualizes timelines, provenance chains, retrieval traces, model traces, evaluation runs, and compressed social state summaries.

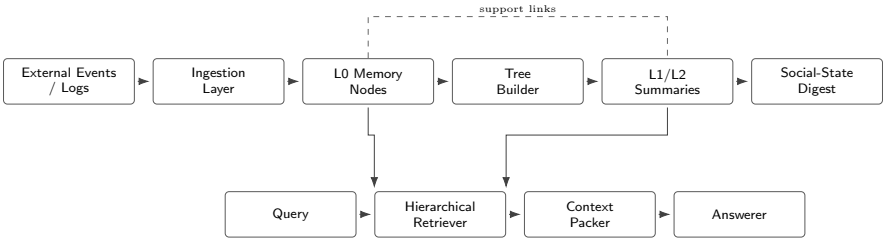

Figure 1: **Core memory architecture for the system.** External events are ingested into L0 episodic memory nodes, which are abstracted into L1 and L2 summaries by the tree builder while preserving support links to underlying evidence. At query time, the hierarchical retriever draws on both raw memories and higher-level summaries to assemble context for answer generation, and the same memory substrate also supports a derived social-state digest.

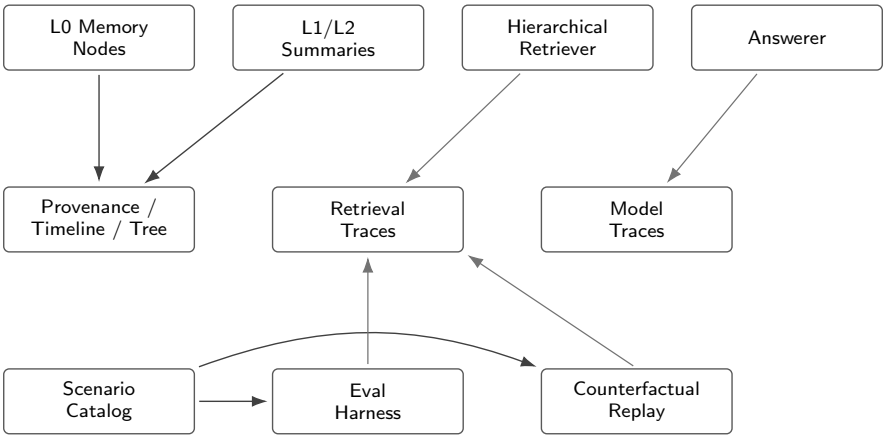

Figure 2: **Observability and evaluation architecture.** The runtime system exposes provenance views over stored memory structure, retrieval traces over branch selection and packed evidence, and model traces over answer-generation behavior. A scenario catalog drives both the evaluation harness and counterfactual replay, which feed retrieval traces and make it possible to inspect how memory policies behave under controlled perturbations and alternative event histories.

4 System Design and Design Choices

Each `MemoryNode` stores text, timestamps, a level within the memory hierarchy, a node type, explicit parent child support relationships, importance and retrieval metadata, structured summary fields, quality status, and source provenance. A summary is a typed object with explicit lineage, support links, versioning behavior, and quality annotations instead of simply generated text.

Flat retrieval is retained as a baseline, while the hierarchical condition clusters related L0 nodes, generates L1 summaries, and stores those summaries with support links to the source events. In the flat baseline, every retrieval decision must be made at the level of individual observations, even when the query targets a longer running social thread or a revised commitment distributed across multiple events. The hierarchical retriever instead routes through summaries when they are sufficient, descends into supporting leaves when detail is required, and assembles a context that is more compressed and more structured than a purely flat alternative.

Provenance is treated as a first class concern because the project aims to support grounded reasoning rather than merely fluent output. Support node identifiers are stored on summaries, parent child edges are explicit, provenance can be queried through dedicated API endpoints, and retrieval and model traces are persisted separately. This same provenance layer also enables stale summary detection and refresh, supports visual inspection in the UI, and enriches counterfactual replay by allowing differences to be computed over comparable retrieved signatures rather than opaque free form outputs alone.

The codebase separates verification from answer generation rather than treating correctness as something a single model call can self certify. Summary generation and summary verification are implemented as distinct components, and answer generation and answer verification are likewise separated. This separation treats compression quality and downstream response quality as related but distinct problems. A summary may be coherent but unsupported, and an answer may be fluent but extend beyond the retrieved evidence. By splitting these stages, the system can evaluate whether a summary faithfully represents its supports and whether an answer remains grounded in retrieved context. This is consistent with the dissertation's broader emphasis on fidelity rather than plausibility alone.

The social state digest is implemented as a derived abstraction rather than as a direct rendering of raw memory because raw episodic recall is often an awkward interface for behaviorally meaningful state. The digest builder in `social_state.py` traverses active, non stale memories and compresses them into categories such as active commitments, active revisions, relationship guidance, open tensions, and likely next actions. Wherever possible, it uses structured summary fields already present on nodes; where such fields are absent, it falls back to explicit heuristics over node text. Each resulting item retains provenance through its support node identifiers, yielding a representation that is more useful for inspection and decision making than a simple list of raw events while still preserving an audit trail.

Counterfactual replay serves a related purpose. If the memory substrate is to support more than retrospective recall, it must be able to show how changes in observations alter downstream reasoning. The counterfactual replay module therefore takes a seeded scenario, applies one or more local edits, replays both the base and variant cases through fresh service instances, and compares their outputs in terms of retrieval depth, token usage, branch count, summary count, retrieved signatures, and answer change. This treats memory not merely as a repository of past events, but as a manipulable substrate for behavioral comparison and intervention analysis to increase its utility.

The current design also reflects several deliberate tradeoffs and simplifications. Summary construction is currently focused on the L0 to L1 case, even though the schema supports deeper hierarchies and the demo data includes L2 summaries. Clustering and social state extraction include heuristic elements rather than learned latent state modeling. Planning is represented structurally in the schema and conceptually in the surrounding architecture, but the repository does not yet implement a full autonomous planning and action loop. The evaluation scenarios are seeded and targeted rather than open world, and the model provider can fall back to mock behavior for local experimentation.

The codebase uses typed shared schemas, a modular service boundary, explicit persistence models, migration backed schema evolution, and stable API surfaces. Retrieval traces, model traces, provenance endpoints, evaluation outputs, and inspection views are all treated as premier system artifacts. The repository is also backed by an extensive automated test suite covering retrieval behavior, memory refresh, evaluation runs, batch ingestion, counterfactual replay, social state derivation, and boundary conditions in the retrieval pipeline.

5 Evaluation Design

The evaluation strategy for this project was designed to test the memory architecture as a retrieval system. The central hypothesis was that a hierarchical, provenance preserving memory structure would outperform flat episodic retrieval on scenarios in which relevant evidence is distributed across time, obscured by routine noise, or requires resolving revisions, relationship context, or interference among similar entities.

To test that hypothesis, the system was evaluated on a seeded scenario catalog spanning recurring retrieval challenges such as delayed commitments, routine interruption, relationship context, commitment revision, and entity interference. These scenarios were chosen because they directly pressure the properties the memory hierarchy is intended to improve: long horizon recall, thread level synthesis, disambiguation, and the preservation of updated or socially specific context.

The strongest quantitative evaluation comes from the frontier sweep reported in `reports/frontier_mvp_rigorous.json`. Rather than evaluating a single hand tuned retrieval policy, this sweep explored a structured policy search space over 49 candidate configurations using Latin hypercube sampling. The

search ranged over 26 retrieval policy parameters, including branch expansion thresholds, disambiguation margins, low confidence handling, and supplemental selection weights. Three optimization runs were performed with random seeds 7, 8, and 9 in order to test whether useful retrieval strategies were stable across search trajectories rather than artifacts of a single run.

The optimization objective was intentionally multi slice. Candidate policies were evaluated across three weighted scenario slices: canonical scenarios with standard seeds, unseen seeds that tested generalization beyond the initial benchmark instances, and hard perturbations that introduced paraphrase and noise variants such as concise wording, indirect phrasing, colloquial expression, typographic noise, altered word order, and distractor entity swaps. The slice weights in the report assign 0.50 of the objective mass to canonical scenarios, 0.25 to unseen seeds, and 0.25 to hard perturbations. This design ensures that the retrieval policy is not rewarded merely for fitting the original benchmark phrasing but must also remain effective under lexical and seed level variation.

The sweep also included family level objectives tied to specific failure modes. In particular, it maximized slot gain on the `multi_person_interference` family and token delta performance on the `time_window_pressure` family. These objectives ensure that evaluation rewards both improvement and robustness on tasks that specifically stress entity disambiguation and budget sensitive retrieval; the architecture is not judged solely by whether it wins on average, but by whether it improves under the kinds of ambiguity and compression pressure that motivate hierarchical memory in the first place.

The primary metrics tracked in the sweep were slot gain, keyword gain, hierarchy win rate, and token delta, along with robust worst case measures and flat win penalties. These metrics were appropriate to the project's scope because they directly capture whether the hierarchical policy retrieves more of the behaviorally relevant content than the flat baseline, and whether it does so within a reasonable token budget. Success, therefore, was defined as improved recall under structural perturbation, with stable behavior across multiple search seeds and without catastrophic regressions against the flat baseline.

6 Results

The frontier sweep provides strong evidence that the hierarchical retrieval architecture improves targeted recall over the flat memory baseline. Across all 49 evaluated candidates, the global mean slot gain was 0.2906 and the global mean keyword gain was 0.2979, while the global mean token delta was -9.30 (where token delta is defined as `baseline_tokens` - `hierarchy_tokens`). This is an expected tradeoff for a system that sometimes chooses to descend into supporting evidence rather than rely only on shallow top ranked episodic items.

The sweep produced a Pareto frontier of six candidates, indicating that the search uncovered multiple viable retrieval regimes rather than a single universally dominant setting. The strongest frontier candidate, `candidate_0028`, achieved a utility score of 0.965, with a slot gain of 0.295, a keyword gain of

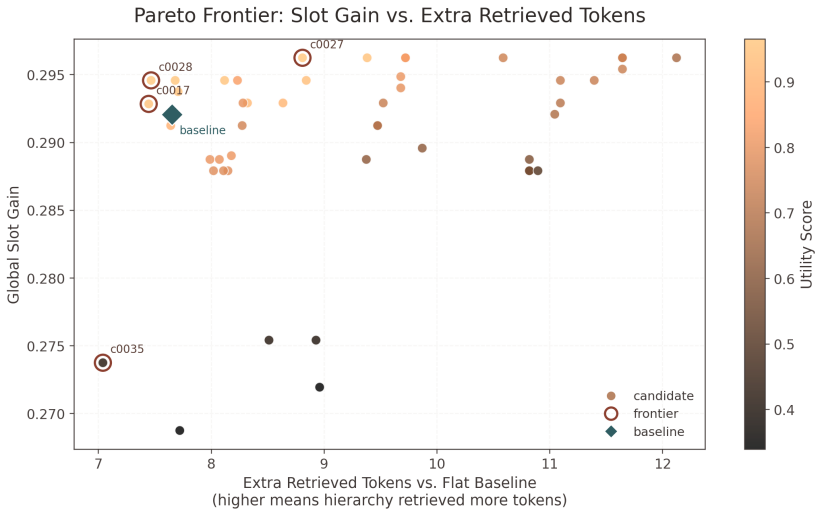

Figure 3: **Pareto frontier over recall gain and retrieval cost.** Each point is a sampled retrieval policy from the frontier sweep, colored by overall utility score. The circled points mark the local Pareto frontier in the displayed space, while the baseline is shown separately for reference. The figure shows that the strongest hierarchical policies improve slot recall beyond the flat baseline, but typically do so by retrieving and packing additional evidence.

0.302, a hierarchy win rate of 0.662, and a token delta of -7.467. A closely related frontier candidate, `candidate_0027`, slightly improved the slot and keyword gains to 0.296 and 0.304 and increased the win rate to 0.667, but did so at a somewhat higher token cost of -8.808. Together, these two candidates show that the retrieval hierarchy does not admit only a single best setting, but rather a small family of strong operating points that trade recall against retrieval compactness.

The remaining frontier candidates reinforce that pattern. `candidate_0040` and `candidate_0017` occupy similar high utility regions, again suggesting a stable high performing family rather than a one off outlier. At the lower end, `candidate_0035` and `candidate_0018` remain non dominated because they occupy different tradeoff positions, though they perform materially worse than the top group.

The win-rate figures should be interpreted carefully. A hierarchy win rate in the 0.66 range does not mean that the flat baseline wins the remaining third of cases. In the strongest frontier configurations, flat wins are nearly absent: for `candidate_0028`, the flat win rate is only 0.004 while the tie rate is 0.333, and for `candidate_0027`, the flat win rate is 0.000 while the tie rate remains 0.333. When hierarchy does not produce a strict improvement, it is therefore usually matching the flat retriever rather than losing to it.

The robustness analysis is equally informative. The stability analysis identified four recurring policy modes across the three optimization runs, with an average pairwise mode Jaccard of 0.639. Two modes, `mode_01` and `mode_02`, appeared in every run. `mode_01` had a centroid slot gain of approximately 0.294, a keyword gain of 0.301, a hierarchy win rate of 0.661, and a token delta of -7.693. `mode_02` improved these recall figures slightly, with a slot gain of 0.296, a keyword gain of 0.303, and a win rate of 0.667, but at a larger token cost of -8.836. The persistence of these two modes across all three optimization seeds reflects a stable region of policy space in which hierarchical retrieval consistently outperforms the flat alternative on the chosen objectives.

At the same time, the sweep also shows that the architecture's gains are not cost free and are not uniformly distributed. The negative token delta figures across the frontier indicate that recall improvements are generally purchased by retrieving and packing more evidence. Moreover, the stability report shows that lower performing modes continue to appear in some runs, and the baseline itself appeared on the frontier in the run seeded with 8. This nuance implies that there is no single retrieval policy that dominates across every possible objective weighting, and that the flat baseline can remain competitive under some slices of the search space. Taken together, the results show three things. First, the hierarchical memory structure consistently produces measurable gains over flat episodic retrieval on targeted behavioral memory tasks. Second, those gains remain visible across canonical scenarios, unseen seeds, and harder query perturbations, which suggests that the improvement is not merely lexical overfitting. Third, the architecture exposes a real tradeoff surface between recall and token efficiency, which is precisely the kind of systems level behavior that this project set out to make visible. In that sense, the results validate not only the usefulness

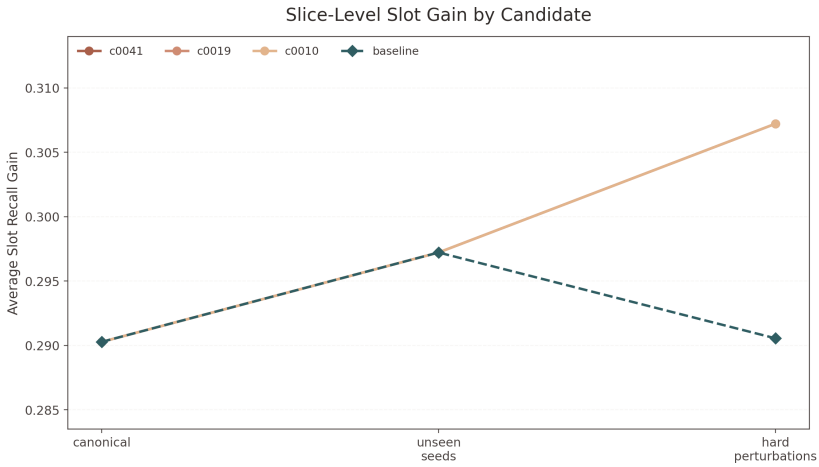

Figure 4: **Slice-level robustness of leading candidates.** Average slot recall gain is shown across canonical queries, unseen seeds, and harder perturbations for three locally selected frontier candidates and the flat baseline. The top candidates remain close to baseline on canonical and unseen settings, but separate more clearly under hard perturbations, indicating that the hierarchy's advantage is most visible when retrieval conditions become less forgiving.

of the hierarchy, but also the broader premise that memory architecture should be treated as an object of empirical optimization rather than as an invisible implementation detail.

7 Limitations

Despite these encouraging results, the project remains a bounded systems prototype rather than a complete simulation platform. The first and most important limitation is that the evaluation scenarios are synthetic and hand authored. They are carefully structured to stress the relevant retrieval and memory behaviors, and they are varied across seeds and perturbations, but they are not equivalent to large scale real world organizational or population data. As a result, the results should be interpreted as evidence about architectural behavior under controlled conditions, not as direct proof of deployment readiness.

A second limitation is scale. The system operates over memory graphs, traces, and evaluation scenarios in a way that is legible and experimentally useful, but it has not been validated under the scale, concurrency, and operational load associated with full organization scale or population scale simulation. Although the storage layer supports PostgreSQL and migration backed persistence, and the service surface is reasonably modular, the current work should still be understood as a rigorously engineered prototype rather than a production scale behavioral simulation stack.

A third limitation concerns the nature of the abstractions themselves. The social state digest is intentionally useful and inspectable, but it is not derived from a learned latent social state model. It combines structured fields from generated summaries with heuristic detection of commitments, revisions, guidance, tensions, and next actions. This makes it practical and legible, but it also means that the digest remains partly rule mediated. It should therefore be interpreted as a compressed view over memory rather than as a complete theory of agent state.

The project also does not yet implement a fully emergent multi agent simulation loop. The dissertation situates memory, reflection, and planning within broader generative agent systems capable of producing large scale social dynamics, whereas this repository focuses on the memory and retrieval substrate beneath such systems. It supports seeded scenarios, counterfactual replay, and derived social state inspection, but it does not yet include a setting in which multiple agents continuously perceive, act, update one another's worlds, and generate emergent collective outcomes over long horizons.

Finally, the evaluation framework is targeted rather than open ended. This is a strength insofar as it makes the architecture measurable and falsifiable, but it also means that the scenarios probe a specific class of structural retrieval challenges rather than the full range of tasks that might matter for a broader behavioral simulation system. The strongest interpretation of the current results is therefore not that the project has solved memory for generative agents, but that it has identified and validated a promising architectural direction under a well defined evaluation regime.

8 Future Work

The most immediate avenue for future work is a richer data ingestion and normalization layer. The current project supports batch event log ingestion with source metadata and idempotent event identifiers, but this path could be extended substantially. A stronger next step would be to ingest heterogeneous experience streams such as transcripts, diaries, meeting logs, or simulated world events, normalize them into a common event schema, and preserve provenance through each transformation step. This would make the memory hierarchy more faithful not only to the dissertation's memory stream framing, but also to the realities of constructing behaviorally grounded agents from messy upstream inputs.

The second direction is explicit snapshotting and replay over evolving event streams. The counterfactual replay module already demonstrates how local edits to a scenario can be pushed through fresh service instances and compared at the level of retrieval and answer outputs. A more complete version of this idea would support memory snapshots over time, partial rollback, branch specific replay, and richer diffing of both summaries and social state abstractions. Such a system would move the architecture closer to a genuine simulation substrate, where interventions can be applied, replayed, and audited across evolving state.

The third natural extension is multi agent interaction on top of the existing memory substrate. At present, the architecture is well suited to modeling an individual agent's evolving memory and social understanding. The next step would be to place multiple such agents into a shared environment in which their observations, plans, and socially derived abstractions influence one another. This would bring the project closer to the dissertation's larger concern with emergent social phenomena, while still preserving the provenance, tracing, and evaluation discipline that make the current implementation valuable.

Future work should also strengthen the evaluation harness itself. The frontier sweep already demonstrates the usefulness of optimizing over multiple slices, perturbations, and robustness criteria. That approach could be extended with broader scenario families, stronger ablation regimes, richer confidence intervals, and more explicit measures of abstraction quality in addition to retrieval quality. In particular, the project would benefit from evaluation tasks that directly measure whether derived abstractions such as social state digests improve downstream planning or intervention decisions rather than merely summarizing the underlying memory more compactly.

Finally, the system could support more effective stakeholder facing interfaces for interacting with agent state. The current inspection UI is oriented toward research and debugging, which is appropriate for the project's present stage. A future version could present intervention oriented dashboards that surface current commitments, tensions, revisions, and plausible next actions in a more decision centric form, while preserving drill down access to provenance.

9 Conclusion

This project set out to test whether hierarchical memory with explicit provenance can better support behaviorally grounded retrieval and abstraction than flat episodic memory alone. The resulting system shows that memory architecture is not merely an implementation detail beneath generative agents, but a meaningful determinant of what such systems can recover, compress, explain, and compare.

The evaluation results support that claim. Across seeded retrieval challenges and policy sweeps, hierarchical retrieval produces consistent improvements in slot and keyword recovery over the flat baseline while exposing explicit tradeoffs between stronger recall and increased token usage. Just as importantly, the system makes those tradeoffs visible through provenance, traces, and structured evaluation rather than burying them inside opaque prompting behavior.

At the same time, the project remains intentionally bounded. It does not yet constitute a full multi agent simulator or a deployment scale behavioral platform. It does, however, provide a memory, retrieval, and abstraction substrate that can support future work on richer ingestion pipelines, evolving event streams, multi agent interaction, and intervention oriented scenario analysis.

Referenced Works

- Park, Soon Jung. *Generative Agent Simulations of Human Behavior*. Dissertation, 2025.
- Haoran Sun and Shaoning Zeng, *Hierarchical Memory for High-Efficiency Long-Term Reasoning in LLM Agents*, arXiv preprint, 2025
- Guibin Zhang, Muxin Fu, Kun Wang, Guancheng Wan, Miao Yu, and Shuicheng Yan, *G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems*, arXiv preprint, 2025